



เอกสารรายงาน

การศึกษา Algorithm สำหรับการทดสอบซอฟต์แวร์

และการออกแบบ Prompt

สำหรับ AI

สมาชิกในกลุ่ม

673380051-7	นางสาวพรชพร	มุสันเทียะ
673380052-5	นางสาวพิตดา	เกษมาลา
673380065-6	นายสรวิษฐ์	ทัตดร
673380280-2	นายปิยพล	ตุนป่า

เสนอ

ผศ. ดร.ชิตสุธา สุ่มเล็ก

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance
ภาคเรียนที่ 1 ปีการศึกษา 2569
สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น
(เดือน กันยายน พ.ศ. 2569)

สารบัญ

เรื่อง	หน้า
สารบัญ	ก
สารบัญตาราง	ค
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของงาน	2
บทที่ 2 ทฤษฎีและเครื่องมือที่เกี่ยวข้อง	3
2.1 Genetic Algorithm (GA)	3
2.2 Feedback-directed Random Testing (FRT)	4
2.3 เครื่องมือที่ใช้ในการทดลอง	6
2.4 Deepseek-flash-v4	8
2.5 Gemini Flash - 3.8	12
บทที่ 3 วิธีการดำเนินการทดลอง	16
3.1 สภาพแวดล้อมและเครื่องมือที่ใช้	16
3.2 ชุดข้อมูล Defects4J	16
3.3 การสร้าง Test Case	16
3.4 การจำแนกผลลัพธ์ Test Case ที่สร้างได้	17
3.5 การกำหนดค่า Configuration	18
3.6 ขั้นตอนการทดลอง	19
3.7 การเก็บข้อมูลและผลการทดลอง	19

บทที่ 4 ผลการทดลองและการวิเคราะห์	20
4.1 ผลการทดลองของ Genetic Algorithm	20
4.2 ผลการทดลองของ Feedback-directed Random Testing	21
4.3 ผลการทดลองของ Deepseek-flash-v4	22
4.4 ผลการทดลองของ Gemini	23
4.5 การเปรียบเทียบผลการทดลองทั้ง 4 แนวทาง	24
4.6 การวิเคราะห์ข้อดีและข้อจำกัดของแต่ละแนวทาง	24
4.7 สรุปผลการเปรียบเทียบ	26
บทที่ 5 สรุปผล ปัญหา และข้อเสนอแนะ	28
5.1 สรุปผลการทดลอง	28
5.2 ปัญหาและอุปสรรคในการทดลอง	28
5.3 สิ่งที่ได้เรียนรู้และแนวทางพัฒนาในอนาคต	29
เอกสารอ้างอิง	30

สารบัญตาราง

ตารางที่ 1 ตารางเครื่องมือที่ใช้ในการทดลอง	6
ตารางที่ 2 ตารางสรุปผลการทดลองของGenetic Algorithm	20
ตารางที่ 3 ตารางสรุปผลการทดลองของFeedback-directed Random Testing	21
ตารางที่ 4 ตารางสรุปผลการทดลองของDeepseek-flash-v4	22
ตารางที่ 5 ตารางสรุปผลการทดลองของ Gemini	23
ตารางที่ 6 เปรียบเทียบ Average Line coverage และ Average Condition coverage ของเครื่องมือทั้ง 4 ตัว	24
ตารางที่ 7 ตารางเปรียบเทียบการการจำแนกสถานะผลลัพธ์ของเครื่องมือทั้ง 4 ตัว	24

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

การทดสอบซอฟต์แวร์ (Software Testing) เป็นกระบวนการสำคัญในการตรวจสอบคุณภาพของซอฟต์แวร์ ค้นหาข้อผิดพลาด และตรวจสอบการทำงานของโปรแกรมให้เป็นไปตามที่กำหนด อย่างไรก็ตาม การสร้าง Test Case จำนวนมากด้วยวิธีทั่วไปอาจใช้เวลาและทรัพยากรสูง จึงมีการนำเทคนิคการสร้าง Test Case แบบอัตโนมัติ (Automated Test Case Generation) และเทคโนโลยีปัญญาประดิษฐ์ (Artificial Intelligence: AI) เข้ามาช่วยในการสร้างและปรับปรุง Test Case

โครงการนี้ศึกษาการสร้าง Test Case โดยใช้ Genetic Algorithm (GA) และ Feedback-directed Random Testing (FRT) ซึ่งเป็นแนวทางที่สามารถนำมาประยุกต์ใช้ในการทดสอบซอฟต์แวร์ โดย GA ใช้แนวคิดการวิวัฒนาการและ Fitness Function เพื่อค้นหาและปรับปรุง Test Case ส่วน FRT ใช้การสุ่มร่วมกับ Feedback จาก Test Case ที่สร้างและผลการทดสอบก่อนหน้า นอกจากนี้ยังศึกษา Deepseek และ Gemini ในการช่วยสร้าง Test Case โดยใช้ Prompt ที่ออกแบบให้เหมาะสมกับงานทดสอบซอฟต์แวร์

ในการทดลองใช้ Defects4J เป็นชุดข้อมูลสำหรับทดสอบโปรแกรม Java ที่มีข้อผิดพลาดที่ทราบอยู่แล้ว โดยนำ Test Case ที่สร้างจากแต่ละแนวทางมาทดลองรันและเก็บผลลัพธ์ เช่น Code Coverage และความสามารถในการตรวจจับ Bug เพื่อนำมาเปรียบเทียบ วิเคราะห์ข้อดี ข้อจำกัด และปัญหาที่พบจากการทดลอง

1.2 วัตถุประสงค์

1. เพื่อศึกษาและทดลองใช้ Genetic Algorithm (GA) และ Feedback-directed Random Testing (FRT) ในการสร้าง Test Case แบบอัตโนมัติ
2. เพื่อศึกษาและทดลองใช้ Deepseek-flash-v4 และ Gemini ในการช่วยสร้าง Test Case สำหรับการทดสอบซอฟต์แวร์
3. เพื่อประเมินผลการสร้างและการทำงานของ Test Case จากแต่ละแนวทาง โดยพิจารณาจากผลการทดสอบ เช่น Code Coverage และความสามารถในการตรวจจับ Bug
4. เพื่อเปรียบเทียบผลลัพธ์ของ Automated Test Case Generation Algorithms และ AI-Assisting Tools พร้อมวิเคราะห์ปัญหาและข้อจำกัดที่พบจากการทดลอง
5. เพื่อจัดเก็บ Source Code, Test Code, Prompt, Configuration และผลการทดลองในรูปแบบที่สามารถตรวจสอบและทำการทดลองซ้ำได้

1.3 ขอบเขตของงาน

1. ศึกษาและทดลองใช้ Genetic Algorithm (GA) และ Feedback-directed Random Testing (FRT) เป็น Automated Test Case Generation Algorithms
2. ใช้ Deepseek-flash-v4 และ Gemini เป็น AI-Assisting Tools สำหรับช่วยสร้าง Test Case
3. ใช้ Defects4J เป็นชุดข้อมูลสำหรับการทดลองกับโปรแกรม Java ที่มีข้อผิดพลาดที่กำหนดไว้

4. สร้างและทดสอบ Test Case จากทั้ง Automated Test Case Generation Algorithms และ AI-Assisting Tools
5. เก็บและวิเคราะห์ผลการทดลอง เช่น จำนวน Test Case, Code Coverage, ผลการตรวจจับ Bug และเวลาที่ใช้ในการทดลอง
6. เปรียบเทียบผลลัพธ์ของแต่ละแนวทาง และวิเคราะห์ปัญหา ข้อจำกัด และสิ่งที่เรียนรู้จากการทดลอง
7. จัดเก็บ Source Code, Test Code, Prompt, Configuration และผลลัพธ์ทั้งหมดไว้ใน Repository เพื่อให้สามารถตรวจสอบและทำการทดลองซ้ำได้

บทที่ 2

ทฤษฎีและเครื่องมือที่เกี่ยวข้อง

โครงงานนี้ศึกษาอัลกอริทึมสำหรับการสร้าง Test Case อัตโนมัติ จำนวน 2 แนวทาง ได้แก่ Genetic Algorithm (GA) และ Feedback-directed Random Testing (FRT) โดยแต่ละแนวทางมีหลักการในการสร้างและปรับปรุง Test Case ที่แตกต่างกัน เพื่อใช้ในการทดสอบซอฟต์แวร์และนำผลลัพธ์มาเปรียบเทียบและวิเคราะห์

2.1 Genetic Algorithm (GA)

Genetic Algorithm (GA) เป็นอัลกอริทึมสำหรับการค้นหาและเพิ่มประสิทธิภาพ โดยเลียนแบบกระบวนการวิวัฒนาการทางธรรมชาติ เพื่อนำมาใช้ค้นหาคำตอบที่เหมาะสมจากคำตอบหลายชุด ในงาน Software Testing สามารถนำ GA มาประยุกต์ใช้เพื่อค้นหาและปรับปรุง Test Case ให้สามารถครอบคลุมส่วนต่าง ๆ ของโปรแกรมได้มากขึ้น

หลักการทำงาน

GA จะเริ่มจากการสร้างคำตอบหลายชุดหรือ Population ซึ่งในงาน Software Testing สามารถแทนด้วยชุดของ Test Case จากนั้นประเมินคุณภาพของแต่ละ Test Case ด้วย Fitness Function แล้วเลือก Test Case ที่มีคุณภาพเหมาะสมมาสร้าง Test Case รุ่นใหม่ผ่านกระบวนการ Crossover และ Mutation

กระบวนการดังกล่าวจะทำซ้ำหลายรอบ โดยแต่ละรอบจะนำผลการประเมินจากรุ่นก่อนหน้ามาใช้ในการสร้างรุ่นใหม่ จนกว่าจะถึงเงื่อนไขหยุดที่กำหนด

ขั้นตอนการทำงาน

1. สร้าง Population หรือชุด Test Case เริ่มต้น
2. ประเมินค่า Fitness ของ Test Case แต่ละรายการ
3. เลือก Test Case ที่มีค่า Fitness เหมาะสม
4. นำ Test Case ที่เลือกมาผสมกันด้วย Crossover
5. ปรับเปลี่ยนค่าบางส่วนแบบสุ่มด้วย Mutation
6. สร้าง Population รุ่นใหม่
7. ประเมิน Fitness ของ Test Case รุ่นใหม่
8. ทำซ้ำกระบวนการจนถึงเงื่อนไขหยุดที่กำหนด

การประยุกต์ใช้กับ Software Testing

GA สามารถนำมาใช้ในการสร้าง Automated Test Cases โดยค้นหา Input ที่ช่วยเพิ่มความครอบคลุมของโปรแกรม เช่น Branch Coverage, Statement Coverage และ Path Coverage รวมถึงใช้ค้นหา Test Case ที่สามารถทำให้โปรแกรมเข้าสู่ส่วนของโค้ดที่ต้องการทดสอบได้

ในการสร้าง Test Case ระบบจะเริ่มจาก Population ของ Test Input จากนั้นนำ Test Input ไปทดสอบกับโปรแกรมและคำนวณค่า Fitness เช่น Code Coverage หรือค่าที่ใช้วัดความใกล้เคียงกับเป้าหมายที่ต้องการ จากนั้นเลือก Test Case ที่มี Fitness เหมาะสมมาสร้าง Test Case รุ่นใหม่ด้วย Crossover และ Mutation

ข้อดี

- สามารถค้นหา Test Case ที่เหมาะสมโดยใช้ Fitness Function เป็นตัวกำหนดเป้าหมาย
- สามารถปรับปรุง Test Case ในแต่ละรุ่น ทำให้มีโอกาสค้นหา Test Case ที่มีคุณภาพดีขึ้น
- สามารถนำไปประยุกต์ใช้กับเป้าหมายด้าน Coverage ได้หลายรูปแบบ
- เหมาะกับปัญหาที่มีพื้นที่การค้นหาขนาดใหญ่ ซึ่งการค้นหาด้วยวิธีตรง ๆ ทำได้ยาก

ข้อจำกัด

- ต้องกำหนด Fitness Function ให้เหมาะสมกับเป้าหมายของการทดสอบ
- มีพารามิเตอร์ที่ต้องกำหนด เช่น Population Size, Mutation และ Crossover ซึ่งส่งผลต่อผลลัพธ์
- การค้นหาอาจใช้เวลานานเมื่อพื้นที่การค้นหามีขนาดใหญ่
- ผลลัพธ์อาจแตกต่างกันในแต่ละรอบ เนื่องจากมีขั้นตอนที่เกี่ยวข้องกับการสุ่ม
- ไม่ได้รับประกันว่าจะสามารถค้นหา Test Case ที่ดีที่สุดหรือครอบคลุมทุกส่วนของโปรแกรมได้

2.2 Feedback-directed Random Testing (FRT)

Feedback-directed Random Testing (FRT) เป็นเทคนิคการสร้าง Test Case แบบสุ่มที่นำข้อมูลหรือผลลัพธ์จากการทดสอบก่อนหน้ามาใช้เป็น Feedback เพื่อช่วยกำหนดการสร้าง Test Case ในรอบถัดไป ทำให้การสุ่มมีข้อมูลจากการทดสอบก่อนหน้ามาช่วยกำกับ

หลักการทำงาน

FRT เริ่มจากการสุ่มสร้าง Test Case หรือ Test Sequence แล้วนำไป Execute กับโปรแกรม จากนั้นระบบจะเก็บข้อมูลจากการทดสอบและนำข้อมูลดังกล่าวมาใช้ประกอบการสร้าง Test Case หรือ Sequence ต่อไป

สำหรับ Randoop ซึ่งเป็นเครื่องมือที่ใช้ในการทดลอง ระบบจะสร้าง Sequence จาก Method และ Input ที่มีอยู่ แล้วนำ Sequence ไป Execute กับโปรแกรม จากนั้นใช้ผลลัพธ์ที่ได้ เช่น พฤติกรรมของโปรแกรมและผลจากการ Execute เพื่อพิจารณา Sequence ที่สามารถนำไปใช้สร้าง Sequence ใหม่ได้

ขั้นตอนการทำงาน

1. สุ่มสร้าง Test Input หรือ Test Sequence
2. นำ Test Case ไป Execute กับโปรแกรม
3. ตรวจสอบผลลัพธ์จากการ Execute
4. วิเคราะห์ข้อมูลที่ได้จากการทดสอบ
5. คัดเลือก Sequence ที่สามารถนำไปใช้ประโยชน์ต่อได้
6. ใช้ Sequence ที่มีอยู่เป็นพื้นฐานในการสร้าง Sequence ใหม่
7. ทำซ้ำกระบวนการจนถึงเงื่อนไขหยุดที่กำหนด
8. นำ Test Case ที่สร้างได้ไปใช้ในการทดสอบและประเมินผล

การประยุกต์ใช้กับ Software Testing

FRT สามารถนำมาใช้ในการสร้าง Automated Test Cases เพื่อค้นหา Exception, Defect และพฤติกรรมที่ผิดปกติของโปรแกรม รวมถึงช่วยสำรวจพฤติกรรมของโปรแกรมในส่วนต่าง ๆ

ในการทดลองนี้ใช้ Randoop ซึ่งเป็นเครื่องมือสำหรับภาษา Java ที่ใช้แนวคิด Feedback-directed Random Testing ในการสร้าง Test Sequence โดยอัตโนมัติ โดย Randoop สามารถนำ Sequence ที่สร้างและดำเนินการแล้วมาใช้เป็นพื้นฐานในการสร้าง Sequence ต่อไป

ข้อดี

- สามารถสร้าง Test Case ได้โดยอัตโนมัติจำนวนมาก
- ไม่จำเป็นต้องกำหนด Test Case ทุกกรณีด้วยตนเอง
- ใช้ผลจากการ Execute ก่อนหน้าเป็นข้อมูลประกอบการสร้าง Sequence ต่อไป
- สามารถค้นพบ Exception และพฤติกรรมของโปรแกรมที่อาจไม่ได้ถูกคาดการณ์ไว้ล่วงหน้า
- เหมาะกับการสร้าง Test Sequence สำหรับโปรแกรม Java ที่มี Method จำนวนมาก

ข้อจำกัด

- ผลลัพธ์มีความเกี่ยวข้องกับการสุ่ม จึงอาจได้ผลลัพธ์แตกต่างกันในแต่ละรอบ
- ไม่ได้รับประกันว่าจะสร้าง Test Case ที่ครอบคลุมทุกส่วนของโปรแกรม
- Test Case ที่สร้างขึ้นอาจมีบางกรณีที่ไม่สามารถตรวจจับ Defect ที่ต้องการได้
- ต้องใช้เวลาในการสร้าง Test Case และประสิทธิภาพอาจขึ้นอยู่กับเวลาที่กำหนดให้เครื่องมือทำงาน
- Test Case ที่สร้างโดยอัตโนมัติอาจต้องมีการตรวจสอบเพิ่มเติมก่อนนำไปใช้งานจริง

การนำ FRT มาใช้ในการทดลอง

ในการทดลองของโครงการนี้ใช้ Randoop ในการสร้าง Test Case กับโปรเจกต์ Java จากชุดข้อมูล Defects4J โดยกำหนด Configuration สำหรับการสร้าง Test Case และกำหนดเวลาในการทำงานของแต่ละการทดลอง จากนั้นนำ Test Case ที่สร้างได้ไป Execute และเก็บผลลัพธ์สำหรับนำไปวิเคราะห์ เช่น ผลการตรวจจับ Bug และผลการทดสอบที่ได้จากแต่ละโปรเจกต์

2.3 เครื่องมือที่ใช้ในการทดลอง

ในการทดลองเกี่ยวกับ Automated Test Case Generation มีการใช้เครื่องมือและเทคโนโลยีที่เกี่ยวข้องดังนี้

รายการ	เครื่องมือ / เทคโนโลยี	การใช้งาน
Algorithm 1	Genetic Algorithm (GA)	ใช้เป็นแนวทางในการสร้างและปรับปรุง Test Case

Test Generation Tool	EvoSuite	ใช้สร้าง Automated Test Case สำหรับโปรแกรม Java โดยใช้แนวคิด Search-Based Software Testing และ Genetic Algorithm
Algorithm 2	Feedback-directed Random Testing (FRT)	ใช้เป็นแนวทางในการสร้าง Test Case โดยอาศัย Feedback จากการทดสอบ
Test Generation Tool	Randoop	ใช้สร้าง Test Sequence สำหรับโปรแกรม Java ตามแนวคิด FRT
Dataset	Defects4J	ใช้เป็นชุดข้อมูลโปรเจกต์ Java ที่มี Bug สำหรับการทดลอง
Programming Language	Java	ภาษาของโปรเจกต์ที่นำมาทดสอบ
Result Collection	Python Scripts / CSV	ใช้รวบรวมและจัดเก็บผลการทดลอง

ตารางที่ 1 ตารางเครื่องมือที่ใช้ในการทดลอง

2.3.1 EvoSuite

EvoSuite เป็นเครื่องมือสำหรับการสร้าง Automated Test Case สำหรับโปรแกรมภาษา Java โดยใช้แนวคิด Search-Based Software Testing และ Genetic Algorithm เพื่อค้นหา Test Suite ที่เหมาะสมกับเป้าหมายของการทดสอบ

ในการทดลองนี้ใช้ EvoSuite เป็นเครื่องมือสำหรับ Genetic Algorithm (GA) เพื่อสร้าง Test Case สำหรับโปรแกรม Java และนำ Test Case ที่สร้างได้ไปใช้ในการทดลองและประเมินผล

2.3.2 Randoop

Randoop เป็นเครื่องมือสำหรับภาษา Java ที่ใช้แนวคิด Feedback-directed Random Testing ในการสร้าง Test Sequence โดยอัตโนมัติ โดยระบบจะสร้าง Sequence จาก Method และ Object ที่มีอยู่ แล้วนำ Sequence ไป Execute และใช้ผลที่ได้เป็นข้อมูลประกอบการสร้าง Sequence ต่อไป

ในการทดลองนี้ Randoop ถูกใช้เป็นเครื่องมือหลักสำหรับ Algorithm Feedback-directed Random Testing (FRT)

2.3.3 Defects4J

Defects4J เป็นชุดข้อมูลสำหรับการวิจัยและทดลองด้าน Software Testing โดยรวบรวมโปรเจกต์ Java ที่มี Bug ที่ทราบอยู่แล้ว และสามารถนำมาใช้ในการประเมินเครื่องมือและเทคนิคการทดสอบซอฟต์แวร์ได้

ในการทดลองนี้ นำโปรเจกต์จาก Defects4J มาใช้เป็น Target Programs สำหรับสร้างและทดสอบ Test Case จาก EvoSuite และ Randoop โดยผลการทดสอบจะถูกนำมาเก็บและวิเคราะห์เพื่อประเมินผลของแต่ละแนวทาง

2.4 Deepseek-flash-v4

2.4.1 หลักการ Prompt Engineering ที่ใช้

การออกแบบ prompt สำหรับให้โมเดลสร้าง JUnit test ในงานวิจัยนี้ใช้หลักการต่อไปนี้ ซึ่งเป็นหลักการทั่วไปของ Prompt Engineering ที่นำมาปรับใช้กับงานสร้างเทสต์อัตโนมัติ

1) กำหนดงานและเป้าหมายให้ชัดเจน

ระบุให้ชัดว่าโมเดลต้องทำอะไรและผลลัพธ์ที่มีเกณฑ์อย่างไร เช่น ต้องสร้างเทสต์ให้ class ที่กำหนด ครอบคลุม branch สำคัญ และได้ไฟล์ที่สมบูรณ์ การบอกเป้าหมายและเกณฑ์ความสำเร็จตรงๆ ช่วยลดความกำกวม โมเดลไม่ต้องเดาเจตนาของผู้ใช้ ผลลัพธ์จึงสม่ำเสมอขึ้น

2) จัดโครงสร้างและใช้ตัวค้น

แบ่ง prompt เป็นส่วนที่มีหน้าที่ต่างกัน เช่น เป้าหมาย ข้อกำหนด บริบท ข้อมูลนำเข้า และรูปแบบผลลัพธ์ และใช้ตัวค้นหรือแท็กแยก "คำสั่ง" ออกจาก "ข้อมูล" การแยกเช่นนี้ทำให้โมเดลไม่สับสนว่าส่วนใดคือสิ่งที่ต้องปฏิบัติตาม และส่วนใดคือโค้ดที่ต้องนำไปวิเคราะห์

3) ใช้ Template และ Placeholder

เขียน prompt เป็นแม่แบบที่มีตัวแปรสำหรับส่วนที่เปลี่ยนไปในแต่ละงาน เช่น ชื่อสคริปต์ และ dependency ของโปรเจกต์ ทำให้ใช้ prompt เดียวกันกับหลาย class ได้ และที่สำคัญคือทุกโมเดลที่นำมาเปรียบเทียบได้รับคำสั่งเหมือนกัน ผลการทดลองจึงเป็นธรรมและทำซ้ำได้

4) คำสั่งแบบมีเงื่อนไข (Conditional Instructions)

กำหนดกฎที่แตกต่างกันตาม dependency ของ Project เช่น การระบุ version Junit การระบุเงื่อนไขช่วยป้องกัน compile error จากการใช้ผิดเวอร์ชัน

5) กำหนดข้อจำกัดทั้งเชิงบวกและเชิงลบ

บอกทั้งสิ่งที่ต้องทำ เช่น ใช้ Plain Java object แทน mock, ตั้งชื่อคลาสตามข้อตกลง, ใช้ package เดียวกับซอร์สโค้ด และสิ่งที่ห้ามทำ เช่น ห้ามใช้ไลบรารีที่โปรเจกต์ไม่มี ข้อจำกัดเหล่านี้ทำให้โค้ดที่ได้คอมไพล์และรันผ่าน Maven ได้ในสภาพแวดล้อมจริง ไม่พียงสิ่งที่ไม่ได้อยู่

6) ฝังความรู้เฉพาะด้านลงในคำสั่ง

นำหลักการทดสอบซอฟต์แวร์ระบุเป็นเกณฑ์โดยตรง ได้แก่ normal case, boundary value, null/empty input, exception path และการทดสอบ loop 0 รอบ 1 รอบ และหลายรอบ

7) ควบคุมคุณภาพของผลลัพธ์

ห้ามใช้ assertion ที่ไม่ตรวจสอบอะไรจริง เช่น `assertTrue(true)` ให้ตรวจสอบ return value และ side effect จริง และห้ามสร้างเทสต์ซ้ำซ้อนที่ต่างกันแค่ค่า literal เพื่อให้ coverage ที่ได้สะท้อนคุณภาพจริง ไม่ใช่แค่จำนวนเทสต์ที่เพิ่มขึ้น

8) กำหนดรูปแบบผลลัพธ์อย่างเข้มงวด

ระบุให้ตอบเป็นโค้ดล้วนโดยไม่มีคำอธิบายหรือ Markdown code fence เพื่อนำผลลัพธ์ไปบันทึกเป็นไฟล์ .java แล้วคอมไพล์ได้ทันที

9) ป้องกันปัญหาผลลัพธ์ถูกตัดกลางคัน

สั่งให้เขียนโค้ดกระชับและจบไฟล์ให้สมบูรณ์ เพราะโมเดลมีข้อจำกัดความยาวของ output หากไฟล์ขาดกลางคัน เช่น ไม่มีวงเล็บปิดคลาส จะคอมไพล์ไม่ผ่านและเสียผลการทดลองทั้งไฟล์

2.4.2 โครง prompt ตัวอย่าง

None

Prompt: สร้าง JUnit Test Suite (JUnit 4 / JUnit 5 Compatible)

🎯 วัตถุประสงค์

สร้าง JUnit test suite สำหรับ Java class ในส่วน `<source_code>` โดยต้อง:

1. **เลือกใช้ Test Framework ตาม Dependency ของโปรเจกต์ (JUnit 4 หรือ JUnit 5)**
2. ครอบคลุม public methods และ logic หลักอย่างกระชับ ไม่สร้าง test cases ซ้ำซ้อน
3. มี test case สำหรับ: normal case, boundary value, null/empty input, exception path
4. ครอบคลุม branch สำคัญ (if/else, switch, loop: 0 รอบ, 1 รอบ, หลายรอบ)
5. ใช้ assertion ที่ตรวจสอบ return value และ side effect จริง หลีกเลี่ยง assertion ปลอม เช่น `assertTrue(true)`
6. โค้ดทั้งหมดต้องเขียนเป็นไฟล์ Java ที่จบสมบูรณ์ 100% ปิดคลาสด้วย `}` เสมอ ห้ามหยุดกลางคัน

SYSTEM SPECIFICATIONS & COMPATIBILITY

1. Framework Selection (เลือกตาม Dependency ของโปรเจกต์)

[FRAMEWORK_SPECIFICATION]

- ****หากเป็น JUnit 4 (ค่าเริ่มต้นของ Defects4J):****
 - ใช้ imports: `org.junit.Test`, `org.junit.Before`, `org.junit.After`
 - ใช้ assertion: `org.junit.Assert.\*` (`assertEquals`, `assertNull`, `assertNotNull`, `assertSame`, `fail`)
 - Exception handling: `@Test(expected = XxxException.class)` หรือ `try { ... fail(); } catch (XxxException e) { ... }`
 - Setup/Teardown: `@Before` / `@After`
 -  ห้ามใช้ JUnit Jupiter / JUnit 5 annotations ในโปรเจกต์ JUnit 4
- ****หากเป็น JUnit 5 (Jupiter):****
 - ใช้ imports: `org.junit.jupiter.api.Test`, `org.junit.jupiter.api.BeforeEach`, `org.junit.jupiter.api.AfterEach`
 - ใช้ assertion: `org.junit.jupiter.api.Assertions.\*` (`assertEquals`, `assertNull`, `assertNotNull`, `assertThrows`)
 - Exception handling: `assertThrows(XxxException.class, () -> { ... });`
 - Setup/Teardown: `@BeforeEach` / `@AfterEach`

2. Language & Dependency Constraints

- ใช้ syntax ที่เข้ากันได้กับโปรเจกต์ (Java 7/8 ขึ้นกับโปรเจกต์)
-  ****ห้ามใช้ Mockito / PowerMock / AssertJ / Hamcrest**** เว้นแต่มีระบุไว้ใน dependency ของโปรเจกต์
-  สำหรับ Mock/Stub/Spy ให้เขียนเป็น Plain Java Objects (Static Inner Classes) ภายใน Test File
-  ****Package declaration ของ Test class ต้องตรงกับ Source Code ที่ให้มา****
-  ตั้งชื่อ Test class ว่า ``<SourceClass>Test``

3. Concise Branch Coverage (กระชับและไม่ซ้ำซ้อน)

- ****เน้น Branch ที่สำคัญ****: เจาะจงกรณีที่กระทบต่อเงื่อนไขการตัดสินใจ (decision branches), ขอบเขตค่า (boundary values), ค่าว่าง/null, และ exception handling
- ****หลีกเลี่ยงการสร้างเทสต์ซ้ำซ้อน (Non-redundant)****: ไม่สร้าง method ย่อยหลายตัวที่ทดสอบพฤติกรรมเดียวกันเพียงแค่เปลี่ยน literal เล็กน้อย
- ****ป้องกันปัญหา Output Truncation****: เขียนโค้ดให้กระชับ เพื่อให้สามารถส่งออกไฟล์ Java ที่สมบูรณ์ครบถ้วน ไม่ติด token output limit

📦 Project Dependencies (pom.xml)

```
```xml
[DEPENDENCY_SPECIFICATION]
```
```

Dependencies เพิ่มเติมของโปรเจกต์:

[signature ของ class ที่ target class เรียกใช้ ถ้ามี]

📄 Source Code ที่ต้องการ Test

```
<source_code>
[full class source code ของ target class]
</source_code>
```

📄 Output ที่ต้องการ

1. ตอบเฉพาะโค้ดภาษา Java ทั้งไฟล์ตั้งแต่ `package ...` จนถึงปีกกาปิด `}` เท่านั้น
2. ****ห้ามมีคำอธิบาย บทนำ สรุป หรือข้อความใดๆ นอกเหนือจากโค้ด Java****

3. ****ห้ามใส่ Markdown Code Fence**** (เช่น ````java` หรือ `````) เพื่อให้บันทึกเป็นไฟล์ .java ได้ทันที

4. ตรวจสอบให้มั่นใจว่าปีกกาเปิด-ปิด `{ }` มีจำนวนครบถ้วนและปิดคลาสสมบูรณ์

2.5 Gemini

2.5.1 หลักการ Prompt Engineering ที่ใช้

1) กำหนดบทบาทและเป้าหมาย

ระบุให้ AI ทำหน้าที่เป็น Senior QA Automation Specialist ซึ่งมีความเชี่ยวชาญด้าน Java Unit Testing บนระบบ Legacy พร้อมกำหนดเป้าหมายในการสร้างชุดทดสอบที่ครอบคลุมทั้ง branch, line และ edge case ในระดับสูงสุด รวมถึงสามารถคอมไพล์และประมวลผลได้จริง การระบุบทบาทช่วยให้โมเดลยึดมาตรฐานการพัฒนาทดสอบระดับมืออาชีพ และใช้เกณฑ์ความสำเร็จของงานเป็นบรรทัดฐานตั้งแต่เริ่มต้น

2) แบ่งขั้นตอนการสร้างออกเป็น 2 ระยะ

สั่งให้วิเคราะห์ logic flow เพื่อจัดทำ test matrix ที่ครอบคลุม Happy Path, Boundary & Edge Cases และ Error & Exception Branches ก่อนเข้าสู่เฟสที่สองซึ่งเป็นการลงมือเขียนโค้ด การกำหนดกระบวนการคิดเป็นลำดับขั้นช่วยลดโอกาสการตกหล่นของกรณีทดสอบสำคัญ

3) ระบุสภาพแวดล้อมและเงื่อนไขความเข้ากันได้

กำหนดให้ใช้ไวยากรณ์ภาษา Java 1.4 ถึง Java 7 และเลือกใช้เฟรมเวิร์ก JUnit 4 หรือ JUnit 3.8.1 ในรูปแบบ extends TestCase เนื่องจากระบบเป้าหมายใน Defects4J เป็นซอฟต์แวร์แบบดั้งเดิม การระบุเวอร์ชันไว้อย่างชัดเจนช่วยป้องกันปัญหาข้อผิดพลาดขณะคอมไพล์อันเกิดจากไวยากรณ์หรือ API ที่ไม่รองรับ

4) กำหนดข้อห้ามเชิงลบอย่างชัดเจน

ไม่อนุญาตให้ใช้ไลบรารีภายนอกเช่น Mockito, AssertJ หรือ Hamcrest แต่ให้เลือกใช้ org.junit.Assert.* ร่วมกับ Plain Java Objects แทน ตลอดจนห้ามละเว้นเนื้อหาโค้ดด้วยข้อความ // TODO และต้องส่งมอบเป็นไฟล์ทดสอบที่ทำงานได้สมบูรณ์ ข้อจำกัดเหล่านี้ช่วยป้องกันไม่ให้ไฟล์โค้ดอ้างอิงถึงไลบรารีที่ไม่มีในระบบ

5) จำกัดขอบเขตการทำงานของ AI

ห้ามประมวลผลคำสั่งใน Terminal ห้ามค้นหาข้อมูลบนเว็บหรือเชื่อมโยงไป URL ภายนอก และห้ามเข้าถึงไฟล์อื่นภายในเครื่อง ให้ดำเนินงานวิเคราะห์เฉพาะจากเนื้อหาโค้ดที่แนบมาเท่านั้น ทั้งนี้เพื่อให้ผลลัพธ์มาจากข้อมูลที่กำหนดอย่างแท้จริง และรักษามาตรฐานสภาพแวดล้อมการทดลองให้สม่ำเสมอทุกครั้ง

6) กำหนดรูปแบบโครงสร้างและการตั้งชื่อ

ให้เรียบเรียงตามรูปแบบ Arrange-Act-Assert (AAA) พร้อมแบ่งแยกเมธอดการทดสอบเดี่ยวสำหรับแต่ละกรณีอย่างชัดเจนและกำหนดรูปแบบชื่อเมธอดเป็น `methodName_givenScenario_shouldExpectedBehavior` ส่งผลให้เข้าใจประสงค์ของการทดสอบได้ทันทีและเอื้อต่อการตรวจสอบ

2.5.2 โครงสร้าง prompt ตัวอย่าง

None

[ROLE & GOAL]

คุณคือ Senior QA Automation Specialist ที่เชี่ยวชาญ Java Unit Testing สำหรับระบบ Legacy Enterprise

เป้าหมาย: สร้าง Complete JUnit Test Suite ที่ครอบคลุม Branch, Line และ Edge Cases Coverage สูงสุด

โดยต้องคอมไพล์และรันผ่านจริงภายใต้สภาพแวดล้อมระบบเดิม (Legacy Build System)

[STRICT SECURITY & EXECUTION CONSTRAINTS]

 กฎเหล็กด้านการทำงาน :

1. ห้ามรันคำสั่ง Terminal / Shell ทุกชนิด (ห้าม git, ls, dir, powershell, find)
2. ห้ามใช้ Web Search, ห้ามเปิด Browser, ห้ามเข้าถึง URL ภายนอกเด็ดขาด
3. ห้ามพยายามสแกนหาไฟล์อื่นในเครื่อง วิเคราะห์จากตัวอักษรในส่วน [SOURCE CODE] ที่แนบมานี้เท่านั้น

[SYSTEM SPECIFICATIONS - STRICT COMPATIBILITY]

- Language Compatibility: Java 1.4 / Java 7 syntax เท่านั้น (ห้ามใช้ diamond operator <>, var, lambda, streams)
- Test Framework: JUnit 4 (หรือ JUnit 3.8.1 สไตล์ extends TestCase)
 - * ใช้ org.junit.Test, org.junit.Assert.*
- Mocking & Assertion Libraries: ห้ามใช้ภายนอกเด็ดขาด (ห้าม Mockito, AssertJ, Hamcrest) ให้ใช้ Standard org.junit.Assert.* และ Plain Java Objects เท่านั้น
- Package: Package declaration ของ Test class ต้องตรงกับ Source Code ที่ให้มา

[SOURCE CODE]

// วาง Source Code ของไฟล์ Java ที่ต้องการให้ทดสอบที่นี่ (เช่น โค้ดของ SoundexUtils.java)

[STEP-BY-STEP INSTRUCTIONS]

Phase 1 - Code Analysis & Test Matrix:

วิเคราะห์ Logic Flow จากตัวโค้ดที่ให้ด้านบน ทำรายการ Test Cases แบ่งเป็น:

- Happy Path
- Boundary & Edge Cases (null, empty string, boundary index, off-by-one)
- Error & Exception Branches (ใช้ @Test(expected = ...) หรือ try-catch คู่กับ fail())

Phase 2 - Test Suite Implementation:

- เขียนโค้ด Unit Test เติมรูปแบบ พร้อม imports ครบถ้วน
- ใช้โครงสร้าง Arrange-Act-Assert (AAA)
- แยกเป็น method @Test เดี่ยวๆ ให้ตรวจสอบง่าย

[STRICT CONSTRAINTS]

- ห้ามใช้ // TODO หรือตัดทอนโค้ดใดๆ ต้องเป็น Fully Runnable Code
- Convention การตั้งชื่อ:
methodName_givenScenario_shouldExpectedBehavior
- ตอบกลับมาเฉพาะผลการวิเคราะห์และ Code Block ของ Test Class เพียงไฟล์เดียวเท่านั้น

บทที่ 3

วิธีการดำเนินการทดลอง

3.1 สภาพแวดล้อมและเครื่องมือที่ใช้

การทดลองดำเนินการบน Windows 11 โดยใช้ WSL Ubuntu สำหรับเรียกใช้ Defects4J และสคริปต์ที่ต้องทำงานในสภาพแวดล้อม Linux ใช้ Java 11 สำหรับคอมไพล์และทดสอบโปรแกรม และใช้ Python 3.14 สำหรับสคริปต์เตรียมข้อมูล สร้าง Test Case และรวบรวมผล ส่วนการพัฒนาและตรวจสอบโค้ดใช้ Antigravity IDE และ Visual Studio Code

เครื่องมือสร้าง Test Case ที่ศึกษาแบ่งเป็น 4 วิธี ได้แก่

1. Gemini 3.8 Flash
2. DeepSeek V4 Flash
3. Randoop 4.3.4
4. EvoSuite 1.2.0

3.2 ชุดข้อมูล Defects4J

ในการดำเนินการทดลองครั้งนี้ ได้เลือกใช้ชุดข้อมูล Defects4J ซึ่งครอบคลุมระบบงานทั้งสิ้น 17 โปรแกรม รวมกับบักเป้าหมายรวม 854 bug ID โดยในแต่ละรายการจะเลือกเฉพาะไฟล์ซอร์สโค้ดในส่วนคลาสที่มีข้อ Bug ตามที่ระบุไว้ จัดเก็บรวบรวมลงในไดเรกทอรี Resouce/ เพื่อใช้เป็น Target Program สำหรับการประมวลผลสร้างชุดทดสอบ ทั้งนี้ กรณีที่ bug ID หนึ่งรายการประกอบด้วยคลาสเป้าหมายหลายคลาส ระบบจะจัดเก็บและบริหารจัดการข้อมูลแยกตามชื่อโปรแกรมพร้อม Bug id อย่างเป็นระบบ

3.3 การสร้าง Test Case

Gemini 3.8 Flash และ DeepSeek V4 Flash

กระบวนการทำงานเริ่มจากการดึงไฟล์ซอร์สโค้ดในระดับคลาสเป้าหมาย เพื่อนำมาประกอบเข้ากับโครงแม่แบบ Prompt ที่เตรียมไว้ จากนั้นจึงส่ง Request ไปยัง KGU GenAI API ผ่านทาง Endpoint `/api/v1/chat/completions` เมื่อระบบประมวลผลและตอบกลับเรียบร้อยแล้ว สคริปต์จะทำการแยกเฉพาะชุดโค้ด Java เพื่อนำไปจัดเก็บบันทึกไว้ในไดเรกทอรี `TestCode/` ของแต่ละโมเดลโดยเฉพาะ ทั้งนี้ ในขั้นตอนการเจนเนอเรตด้วยปัญญาประดิษฐ์ จะมีการบันทึกสถานะการดำเนินงานลงไฟล์ State รายคลาสอย่างเป็นระบบ เพื่อช่วยติดตามความก้าวหน้าและประมวลผลต่อในรายการที่ยังไม่สมบูรณ์ โดยไม่ต้องร้องขอข้อมูลซ้ำในส่วนที่ดำเนินการเสร็จสิ้นแล้ว

Feedback-Directed Random Test Generation

ใช้การสร้าง Test Case แบบ Feedback-Directed Random Test Generation โดยสร้างจากคลาสเป้าหมายในโปรแกรมรุ่น buggy การทดลองแบ่งเป็นสองรอบเพื่อเปลี่ยนค่า seed ของการสุ่ม รอบที่ 1 ใช้ seed '0' และรอบที่ 2 ใช้ seed '20260928' ทั้งสองรอบกำหนดเวลาให้ Randoop สร้าง Test Case เท่ากันที่ 60 วินาทีต่อ bug ID และแยกไฟล์ TestCode กับ state ของแต่ละรอบออกจากกัน

Genetic Algorithm

การทดลองมี 2 รอบ โดยใช้ EvoSuite โหมด STANDARD_GA โดย checkout และคอมไพล์โปรแกรมรุ่น buggy ก่อน แล้วส่งคลาสเป้าหมายและ classpath ให้ EvoSuite สร้างชุดทดสอบ สคริปต์กำหนดเกณฑ์ค้นหาเป็น LINE_BRANCH และบันทึกไฟล์ Java ที่สร้างได้ตามโปรเจกต์และ bug ID ทั้ง 2 รอบ

เหตุผลของการทดลองสองรอบ

Randoop และ EvoSuite เป็นเครื่องมือที่อาศัยการสุ่มในการสร้าง Test Case ผลจากการรันเพียงครั้งเดียวอาจเกิดจากความบังเอิญของการสุ่ม จึงทดลองสองรอบเพื่อตรวจสอบความเสถียรของผล และใช้ผลรวมสองรอบเป็นฐานของการเปรียบเทียบเพื่อลดอิทธิพลของความแปรปรวนจากการสุ่ม

สำหรับ Randoop เปลี่ยนค่า seed ระหว่างรอบ (0 และ 20260928) เพื่อให้ลำดับการสุ่มต่างกัน และตรวจสอบว่าผลไม่ขึ้นกับ seed ใด seed หนึ่ง

สำหรับ EvoSuite คง seed เท่าเดิม (20260918) ทั้งสองรอบ เพื่อวัดความแปรปรวนที่เกิดจากการค้นหาแบบอิงเวลา (search budget 60 วินาที) และการทำงานแบบหลาย thread โดยไม่เปลี่ยนค่าอื่น

ส่วน AI (Gemini และ DeepSeek) สร้างด้วย prompt เดียวกัน การสร้างซ้ำอาจให้ผลต่างกันเนื่องจากธรรมชาติของโมเดลภาษา Gemini มีผลเฉพาะรอบที่ 1 การเปรียบเทียบกับ AI จึงใช้ข้อมูลรอบที่ 1 เป็นหลัก

3.4 การจำแนกผลลัพธ์ Test Case ที่สร้างได้

หลังจากการสร้างชุดทดสอบ (TestCode) เรียบร้อยแล้ว สคริปต์จะนำไฟล์ Java ไป Execute ร่วมกับโปรเจกต์เป้าหมายใน Defects4J โดยเริ่มตรวจสอบการคอมไพล์ของโค้ดทดสอบเป็นลำดับแรก จากนั้นนำชุดทดสอบไปรันเทียบกับทั้งบนซอฟต์แวร์รุ่น buggy และรุ่น fixed เพื่อบันทึกข้อมูล Code Coverage (Line และ Condition Coverage) ปริมาณกรณีทดสอบ ตลอดจนรายการที่ทำงานล้มเหลว แล้วจึงนำผลประเมินมาจำแนกสถานะการทำงานของ Automated Tool และ AI-Assisting Tools ต่อไป

การประเมินผลจะพิจารณาในอัตราส่วน 1 bug ต่อ 1 รอบการทดลอง โดยเริ่มตรวจสอบตั้งแต่กระบวนการสร้าง โค้ดคอมไพล์ การประมวลผล และการวัดความครอบคลุม ก่อนใช้ผลจาก assertion เป็นเกณฑ์ตัดสิน ซึ่งจำแนกออกเป็น 5 สถานะ ดังนี้

1. REVEALING

หมายถึง ชุดทดสอบสามารถค้นพบข้อบกพร่องได้อย่างถูกต้อง โดยการทดสอบล้มเหลวขณะประมวลผลบนเวอร์ชัน buggy เนื่องจากซอฟต์แวร์ยังมี Bug ซ่อนอยู่ แต่สามารถทำงานผ่านเรียบร้อยบนเวอร์ชัน fixed ที่ได้รับการแก้ไขโค้ดแล้ว แสดงว่า TestCase ดังกล่าวสะท้อนถึงจุดผิดพลาดในระบบจริงอย่างมีประสิทธิภาพ ไม่ได้ล้มเหลวจากความผิดพลาดของชุดทดสอบเอง จัดเป็นผลลัพธ์ที่มีสมรรถนะสูงสุด

2. NOT_REVEALING

หมายถึง ชุดทดสอบสามารถประมวลผลผ่านได้ทั้งบนเวอร์ชัน buggy และ fixed แต่ไม่สามารถตรวจพบหรือระบุข้อผิดพลาดที่แฝงอยู่ได้ การทดสอบรูปแบบนี้แม้จะมีประโยชน์ในการตรวจสอบพฤติกรรมพื้นฐานและช่วยเพิ่มค่า Coverage แต่ยังไม่สามารถใช้สำหรับตรวจจับ Defect เป้าหมายได้

3. INCONCLUSIVE

หมายถึง ชุดทดสอบสามารถคอมไพล์และประมวลผลได้ทั้งสองเวอร์ชัน แต่ผลลัพธ์ที่ได้ยังไม่ชัดเจนเพียงพอในการยืนยันความถูกต้องของการตรวจจับบั๊ก เช่น เกิดการล้มเหลวขณะรันบนรุ่น fixed (ทั้งกรณีล้มเหลวทั้งคู่ หรือรันผ่านบน buggy แต่กลับล้มเหลวบน fixed) ซึ่งอาจเกิดจากปัจจัยอื่นที่ไม่เกี่ยวข้องกับ Defect หลัก จึงไม่สามารถสรุปผลประเมินได้อย่างแน่ชัด

4. NOT_AVAILABLE

หมายถึง มีการGenerate Test Case ขึ้นมา แต่ไม่สามารถประมวลผลจนได้ผลลัพธ์เปรียบเทียบระหว่างรุ่น buggy และ fixed ได้ เช่น เกิดข้อผิดพลาดขณะคอมไพล์จากการเรียกใช้ API หรือ External Library ที่ไม่มีในระบบ การประมวลผลถูกขัดจังหวะ ไม่สามารถจับเก็บค่า Coverage ได้ หรือรันสำเร็จเพียงเวอร์ชันเดียว สถานะนี้ประเมินในระดับ Bug ID ซึ่งกรณีที่โค้ดบางคลาสคอมไพล์ไม่ผ่าน อาจส่งผลให้ชุดทดสอบของ Bug นั้นไม่สามารถรันได้ทั้งหมด แต่ไม่ได้หมายความว่าทั้งโปรเจกต์จะล้มเหลว

5. FAIL

หมายถึง อัลกอริทึมสร้างเทสต์อัตโนมัติหรือโมเดล AI ไม่สามารถสร้างไฟล์ทดสอบที่นำมาใช้งานได้สำหรับ Bug ID นั้น (ไม่มีโครงสร้างเมธอด JUnit test หรือกระบวนการเจเนอเรตครั้งล่าสุดเกิดข้อผิดพลาด แม้จะมีไฟล์ทดลองเดิมค้างอยู่ก็ตาม) ทั้งนี้ สถานะ FAIL ไม่ได้หมายถึง Assertion ของ TestCase ล้มเหลว ส่วนกรณีที่สร้างโค้ดได้เพียงบางส่วนของ Bug ID จะไม่จัดเป็น FAIL แต่จะบันทึกเป็นข้อมูลการสร้างแบบไม่สมบูรณ์ (partial generation) แล้วตัดสินสถานะจากผลการประมวลผลเท่าที่มี

หมายเหตุการวัด Coverage: การคำนวณค่า Coverage จะพิจารณาจาก Target Class ของซอฟต์แวร์รุ่น buggy เป็นหลัก หากสามารถวัดค่าในฟังก์ชัน buggy ได้ จะนำค่านั้นมาใช้งานทันทีแม้ผลลัพธ์สรุปจะเป็น NOT_AVAILABLE จากการรันฟังก์ชัน fixed ไม่ผ่าน ส่วนกรณีที่ไม่สามารถเก็บค่า Coverage ฟังก์ชัน buggy ได้ จะถูกคิดเป็น 0% ในภาพรวมการประเมิน และเคสที่มีสถานะเป็น FAIL จะไม่มีการนำข้อมูล Coverage เก่ามาคำนวณซ้ำ

3.5 การกำหนดค่า Configuration

3.5.1 Gemini 3.8 Flash

นำซอร์สโค้ดตั้งต้นจากไดเรกทอรี Resource/ แทรกลงใน Prompt แล้วส่งประมวลผลไปยัง KKU GenAI API โดยระบุชื่อโมเดลเป็น Gemini 3.8 Flash พร้อมทั้งยืนยันสิทธิ์ด้วย API Key ของระบบ KKU GenAI

3.5.2 DeepSeek V4 Flash

นำซอร์สโค้ดจาก Resoucre/ ประกอบเข้ากับ Prompt และส่งไปยัง KKU GenAI API โดยกำหนดการใช้งานผ่านโมเดล DeepSeek V4 Flash ร่วมกับการยืนยันตัวตนด้วย API Key ของระบบ

3.5.3 Feedback-directed Random Testing

ในการทดลองกำหนดการทำงานเป็น 2 รอบ โดยรอบแรกใช้ค่า Seed เท่ากับ 0 และรอบที่สองเปลี่ยนค่า Seed เป็น 20260928 ทั้งนี้ตั้งค่า Time Limit ไว้ที่ 60 วินาที และกำหนด Project Timeout ไว้ที่ 600 วินาที

สำหรับการทดสอบรอบที่สอง ใช้การสุ่มด้วย Seed หมายเลข 20260928 โดยยังคงใช้ระยะเวลา Time Limit 60 วินาที และ Project Timeout 600 วินาทีเช่นเดียวกัน

3.5.4 Genetic Algorithm

ในการทดลองใช้ EvoSuite ในโหมด STANDARD_GA โดยกำหนดเป้าหมายการค้นหาเป็น LINE_BRANCH ตั้งเวลาการทำงาน (Search Budget) ไว้ที่ 60 วินาที และใช้ค่า Random Seed หมายเลข 20260918 เหมือนกันทั้งสองรอบ ซึ่งการทดลองทั้งสองรอบเป็นการดำเนินงานซ้ำภายใต้สถานะเดิม ไม่ได้ปรับเปลี่ยนค่า seed เช่นเดียวกับ FRT เนื่องจาก EvoSuite ยุติกระบวนการค้นหาตามระยะเวลาที่กำหนดและประมวลผลแบบ Multithreading ส่งผลให้ชุดทดสอบที่ได้รับจากการทดลองอาจมีความแตกต่างกันในแต่ละรอบ (ตัวอย่างเช่น โปรเจกต์ Compress_16 สามารถสร้างกรณีทดสอบได้ 21 รายการในรอบที่ 1 และ 20 รายการในรอบที่ 2)

3.6 ขั้นตอนการทดลอง

1. เตรียมรายการโปรเจกต์และ bug ID พร้อมซอร์สโค้ดของคลาสเป้าหมายใน 'Resoucre/'
2. กำหนดค่าเครื่องมือ prompt, seed, budget และตำแหน่งจัดเก็บไฟล์ตามวิธีทดลอง
3. รันสคริปต์สร้าง Test Case และบันทึกไฟล์ Java ลงใน TestCode ของแต่ละวิธีและแต่ละรอบ
4. บันทึกสถานะการสร้าง เพื่อตรวจสอบรายการที่เสร็จแล้วและรายการที่ต้องดำเนินการใหม่
5. นำ Test Case ไปคอมไพล์และรันกับ Defects4J รัน buggy และ fixed
6. วัดความครอบคลุมของโค้ดในรุ่น buggy และรวบรวมผลรายเป้าหมาย
7. รวมข้อมูลจากแต่ละเป้าหมายเป็นรายงานของแต่ละวิธีและแต่ละรอบ

3.7 การเก็บข้อมูลและผลการทดลอง

ผลรายเป้าหมายถูกบันทึกเป็นไฟล์ result.csv ในแต่ละโฟลเดอร์ผลลัพธ์ เก็บประกอบด้วย

1. ข้อมูลระบุการทดลอง ได้แก่ ชื่อโปรเจกต์ หมายเลข bug วิธีและรอบการสร้าง ค่า seed และ budget

2. ข้อมูลการสร้างและการรัน ได้แก่ จำนวน Test Case ที่สร้างได้ เวลาที่ใช้สร้าง ผลการคอมไพล์และการรันบนรุ่น buggy และ fixed จำนวน Test Case ที่ผ่านและที่ล้มเหลว
3. ข้อมูลความครอบคลุม ได้แก่ Line Coverage และ Condition/Branch Coverage ของ target class บนรุ่น buggy
4. สถานะผลลัพธ์ตามการจำแนกในหัวข้อ 3.4 และข้อมูลเสริมกรณีสร้างเทสต์ได้ไม่ครบ (partial generation)

จากข้อมูลรายเป้าหมาย คำนวณค่าสรุปโดยนับ 1 bug ต่อ 1 รอบ ดังนี้

- **Mean Line Coverage (Measured) และ Mean Condition Coverage (Measured)** คือค่าเฉลี่ยของ Line Coverage และ Condition Coverage เฉพาะ bug ที่วัดค่าได้เป็นตัวเลข
- **Mean Line Coverage (Overall) และ Mean Condition Coverage (Overall)** คือค่าเฉลี่ยที่นับ bug ซึ่งวัด Coverage ไม่สำเร็จหรือสร้างเทสต์ไม่ได้เป็น 0% และตัด bug ที่วัดสำเร็จแต่ไม่มี line Coverage หรือ Condition Coverage (N/A) ออกจากตัวหาร

บทที่ 4
ผลการทดลองและการวิเคราะห์

4.1 ผลการทดลองของ Genetic Algorithm

| หมวดหมู่ตัวชี้วัด | Round 1 | Round 2 | All rounds | หมายเหตุ / หน่วย |
|---|---------|---------|------------|-------------------------|
| ภาพรวมการทดสอบ (Test Execution) | | | | |
| Tests generated | 32,789 | 32,711 | 65,500 | จำนวนชุดทดสอบที่สร้าง |
| Tests generate Time | 53,160 | 51,780 | 104,940 | วินาที (Seconds) |
| Total results (max 854) | 853 | 853 | 1,706 | ผลรวมรายการทดสอบ |
| ความครอบคลุมของโค้ด (Code Coverage) | | | | |
| Mean Line Coverage (Measured) | 61.42% | 61.66% | 61.54% | ค่าเฉลี่ย (%) |
| Mean Condition Coverage (Measured) | 53.77% | 54.06% | 53.92% | ค่าเฉลี่ย (%) |
| Mean Line Coverage (Overall) | 58.62% | 59.42% | 59.02% | ค่าเฉลี่ย (%) |
| Mean Condition Coverage (Overall) | 51.30% | 52.09% | 51.70% | ค่าเฉลี่ย (%) |
| การจำแนกสถานะผลลัพธ์ (Test Result Classification) | | | | |
| REVEALING | 0 | 0 | 0 | พบบัก |
| NOT_REVEALING | 602 | 598 | 1,200 | รันผ่าน / ไม่พบบัก |
| INCONCLUSIVE | 191 | 185 | 376 | สรุปผลไม่ได้ |
| NOT_AVAILABLE | 60 | 70 | 130 | No_Run / Compileไม่ผ่าน |

| | | | | |
|------|---|---|---|---------|
| Fail | 1 | 1 | 2 | ล้มเหลว |
|------|---|---|---|---------|

ตารางที่ 2 ตารางสรุปผลการทดลองของ Genetic Algorithm

4.2 ผลการทดลองของ Feedback-directed Random Testing

| หมวดหมู่ตัวชี้วัด | Round 1 | Round 2 | All rounds | หมายเหตุ / หน่วย |
|---|-----------|-----------|------------|-------------------------|
| ภาพรวมการทดสอบ (Test Execution) | | | | |
| Tests generated | 1,463,258 | 1,955,483 | 3,418,741 | จำนวนชุดทดสอบที่สร้าง |
| Tests generate Time | 58,868.59 | 46,732.63 | 105,601.22 | วินาที (Seconds) |
| Total results (max 854) | 847 | 824 | 1,671 | ผลรวมรายการทดสอบ |
| ความครอบคลุมของโค้ด (Code Coverage) | | | | |
| Mean Line Coverage (Measured) | 33.37% | 36.50% | 34.92% | ค่าเฉลี่ย (%) |
| Mean Condition Coverage (Measured) | 23.10% | 26.74% | 24.91% | ค่าเฉลี่ย (%) |
| Mean Line Coverage (Overall) | 32.51% | 35.05% | 33.78% | ค่าเฉลี่ย (%) |
| Mean Condition Coverage (Overall) | 22.51% | 25.67% | 24.09% | ค่าเฉลี่ย (%) |
| การจำแนกสถานะผลลัพธ์ (Test Result Classification) | | | | |
| REVEALING | 1 | 1 | 2 | พบบั๊ก |
| NOT_REVEALING | 580 | 544 | 1,124 | ผ่าน / ไม่พบบั๊ก |
| INCONCLUSIVE | 240 | 264 | 504 | สรุปผลไม่ได้ |
| NOT_AVAILABLE | 26 | 15 | 41 | No_Run / Compileไม่ผ่าน |

| | | | | |
|------|---|----|----|---------|
| Fail | 7 | 30 | 37 | ล้มเหลว |
|------|---|----|----|---------|

ตารางที่ 3 ตารางสรุปผลการทดลองของ Feedback-directed Random Testing

4.3 ผลการทดลองของ Deepseek-flash-v4

| หมวดหมู่ตัวชี้วัด | Round 1 | หมายเหตุ / หน่วย |
|---|------------|-------------------------|
| ภาพรวมการทดสอบ (Test Execution) | | |
| Generation time | 123,382.23 | วินาที (Seconds) |
| Total results (max 854) | 854 | ผลรวมรายการทดสอบ |
| Total tokens | 11,801,383 | token |
| ความครอบคลุมของโค้ด (Code Coverage) | | |
| Mean Line Coverage (Measured) | 73.94% | ค่าเฉลี่ย (%) |
| Mean Condition Coverage (Measured) | 66.09% | ค่าเฉลี่ย (%) |
| Mean Line Coverage (Overall) | 18.70% | ค่าเฉลี่ย (%) |
| Mean Condition Coverage (Overall) | 16.48% | ค่าเฉลี่ย (%) |
| การจำแนกสถานะผลลัพธ์ (Test Result Classification) | | |
| REVEALING | 4 | พบบั๊ก |
| NOT_REVEALING | 38 | ผ่าน / ไม่พบบั๊ก |
| INCONCLUSIVE | 641 | สรุปผลไม่ได้ |
| NOT_AVAILABLE | 172 | No_Run / Compileไม่ผ่าน |

| | | |
|------|---|---------|
| Fail | 0 | ล้มเหลว |
|------|---|---------|

ตารางที่ 4 ตารางสรุปผลการทดลองของDeepseek-flash-v4

4.4 ผลการทดลองของ Gemini

| หมวดหมู่ตัวชี้วัด | Round 1 | หมายเหตุ / หน่วย |
|---|-----------|-------------------------|
| ภาพรวมการทดสอบ (Test Execution) | | |
| Generation time | 23,501.65 | วินาที (Seconds) |
| Total results (max 854) | 854 | ผลรวมรายการทดสอบ |
| Total tokens | 5,376,431 | token |
| ความครอบคลุมของโค้ด (Code Coverage) | | |
| Mean Line Coverage (Measured) | 49.09% | ค่าเฉลี่ย (%) |
| Mean Condition Coverage (Measured) | 43.84% | ค่าเฉลี่ย (%) |
| Mean Line Coverage (Overall) | 28.39% | ค่าเฉลี่ย (%) |
| Mean Condition Coverage (Overall) | 25.27% | ค่าเฉลี่ย (%) |
| การจำแนกสถานะผลลัพธ์ (Test Result Classification) | | |
| REVEALING | 6 | พบบั๊ก |
| NOT_REVEALING | 252 | ผ่าน / ไม่พบบั๊ก |
| INCONCLUSIVE | 363 | สรุปผลไม่ได้ |
| NOT_AVAILABLE | 233 | No_Run / Compileไม่ผ่าน |

| | | |
|------|---|---------|
| Fail | 0 | ล้มเหลว |
|------|---|---------|

ตารางที่ 5 ตารางสรุปผลการทดลองของ Gemini

4.5 การเปรียบเทียบผลการทดลองทั้ง 4 แนวทาง

| Tools | Mean Line Coverage
(Measured) | Mean Line Coverage
(Overall) | Mean Condition
Coverage (Measured) | Mean Condition
Coverage (Overall) |
|--|----------------------------------|---------------------------------|---------------------------------------|--------------------------------------|
| Gemini 3.8 Flash
(Round 1) | 49.00% | 28.39% | 43.76% | 25.27% |
| Deepseek v4 Flash
(Round 1) | 73.94% | 18.70% | 66.09% | 16.48% |
| Genetic Algorithm
(All Round) | 65.02% | 59.02% | 61.37% | 51.70% |
| Feedback-Directed
Random Test Generation
(All Round) | 35.05% | 33.78% | 24.92% | 24.09% |

ตารางที่ 6 เปรียบเทียบ Average Line coverage และ Average Condition coverage ของเครื่องมือทั้ง 4 ตัว

| Verdict | Gemini 3.8 Flash
(Round 1) | Deepseek v4 Flash
(Round 1) | Feedback-Directed
Random Test Generation
(All Round) | Genetic Algorithm
(All Round) |
|---------------|-------------------------------|--------------------------------|--|----------------------------------|
| REVEALING | 0.70% | 0.47% | 0.12% | 0.00% |
| NOT REVEALING | 29.39% | 4.33% | 65.81% | 70.26% |
| INCONCLUSIVE | 27.17% | 20.14% | 29.51% | 22.01% |
| NOT AVAILABLE | 42.51% | 75.06% | 2.40% | 7.61% |
| FAIL | 0% | 0.00% | 2.17% | 0.12% |

ตารางที่ 7 ตารางเปรียบเทียบการการจำแนกสถานะผลลัพธ์ของเครื่องมือทั้ง 4 ตัว

4.6 การวิเคราะห์ข้อดีและข้อจำกัดของแต่ละแนวทาง

1. Gemini 3.8 Flash

ข้อดี

Gemini 3.8 Flash โดดเด่นด้วยสัดส่วน REVEALING ที่สูงที่สุดท่ามกลางทั้ง 4 เครื่องมือ โดยคิดเป็น 0.70% สะท้อนให้เห็นว่าในระดับ Bug ID ของแต่ละรอบการประมวลผล เครื่องมือ Gemini มีศักยภาพในการเจอเนอเรต Test Suite ที่ตรวจพบความผิดพลาดบนซอฟต์แวร์เวอร์ชัน buggy จนเกิดการล้มเหลว แต่กลับประมวลผลผ่านได้เรียบร้อยบนเวอร์ชัน fixed ได้มากกว่าแนวทางอื่น แม้ว่าสัดส่วนดังกล่าวจะยังจัดอยู่ในระดับที่ไม่สูงมากก็ตาม

สำหรับมิติ Code Coverage พบว่า Gemini สามารถทำค่าเฉลี่ย Mean Line Coverage (Measured) ได้ที่ 49.00% และมี Mean Condition Coverage (Measured) อยู่ที่ 43.76% ซึ่งถือเป็นระดับปานกลาง ในขณะที่ค่าเฉลี่ยภาพรวม Mean Line Coverage (Overall) และ Mean Condition Coverage (Overall) มีบันทึกไว้ที่ 28.39% และ 25.27% ตามลำดับ ทั้งนี้ Gemini ไม่พบสภาวะ FAIL (0%) เลยแม้แต่รายการเดียว ชี้ให้เห็นว่าทุก Bug ID ที่นำมาทดสอบ ตัวโมเดลมีความสามารถในการสร้างไฟล์ TestCode ให้มีโครงสร้างโค้ดที่ถูกต้องเพียงพอต่อการนำไปประมวลผลทดสอบในขั้นตอนถัดไป โดยไม่เกิดความล้มเหลวในระดับที่ไม่ได้รับไฟล์ทดสอบเลย

ข้อจำกัด

ประเด็นข้อจำกัดหลักของ Gemini อยู่ที่อัตราส่วน NOT_AVAILABLE ซึ่งพุ่งสูงถึง 42.51% แสดงว่ามี Bug ID จำนวนมากที่ขั้นตอนการวัดผลไม่สามารถดำเนินการต่อเนื่องจากได้ผลลัพธ์การเปรียบเทียบระหว่างรุ่นที่มีข้อผิดพลาดและรุ่นที่ได้รับการแก้ไขอย่างสมบูรณ์ ซึ่งมีสาเหตุมาจากอุปสรรคด้านการคอมไพล์โค้ด ปัญหาการประมวลผล การจัดเก็บค่าความครอบคลุม ตลอดจนการที่ชุดทดสอบรันผ่านได้เพียงเวอร์ชันใดเวอร์ชันหนึ่งเท่านั้น

นอกจากนี้ Gemini ยังมีสัดส่วน INCONCLUSIVE อยู่ที่ 27.17% ซึ่งหมายความว่า Bug ID อีกจำนวนหนึ่งที่ชุดทดสอบประมวลผลและคอมไพล์ผ่านได้ทั้งสองเวอร์ชัน ทว่าผลลัพธ์จากการตรวจสอบ assertion กลับยังไม่อาจนำมายืนยันการตรวจจับ Defect เป้าหมายได้ชัดเจนเพียงพอ เช่น เกิดการล้มเหลวในรุ่น fixed หรือให้ผลที่ไม่ตรงตามเงื่อนไขของ REVEALING และ NOT_REVEALING

ด้วยเหตุนี้ แม้ Gemini จะครองสัดส่วน REVEALING ที่สูงที่สุด แต่ยังคงเผชิญข้อจำกัดเรื่องความสมบูรณ์ในกระบวนการประเมินผลของแต่ละ Bug ID อยู่พอสมควร

2. DeepSeek v4 Flash

ข้อดี

DeepSeek v4 Flash มีระดับความครอบคลุมของโค้ดสูงที่สุดเมื่อวัดเฉพาะกลุ่มที่ประมวลผลสำเร็จ โดยทำค่าเฉลี่ย Mean Line Coverage (Measured) ได้สูงถึง 73.94% และมี Mean Condition Coverage (Measured) อยู่ที่ 66.09% ซึ่งแสดงให้เห็นว่ารายการ Bug ID ที่สามารถรันจนเข้าสู่ขั้นตอนจัดเก็บ Coverage ได้ นั้น ชุดทดสอบที่เจอเนอเรตโดย DeepSeek มีสมรรถนะเข้าถึงคำสั่งและเงื่อนไขต่างๆ ของซอฟต์แวร์ได้อย่างมีประสิทธิภาพสูง

โมเดล DeepSeek ทำอัตราส่วน REVEALING อยู่ที่ 0.47% ซึ่งนับเป็นอันดับสองรองเพียง Gemini และยังเหนือกว่าทั้ง Feedback-Directed Random Test Generation และ Genetic Algorithm อีกทั้งยังคงสถิติปราศจากกรณี FAIL (0%) เช่นเดียวกับ Gemini สะท้อนว่าโมเดลมีความเสถียรในการเจอเนอเรต TestCode สำหรับทุก Bug ID โดยไม่เกิดความล้มเหลวในขั้นตอนการสร้างไฟล์ทดสอบเพื่อนำไปประเมินผล

ข้อจำกัด

จุดบกพร่องสำคัญของ DeepSeek คือมีอัตราส่วน NOT_AVAILABLE สูงสุดในกลุ่มเครื่องมือทั้ง 4 แบบ โดยแต่ละระดับ 75.06% ซึ่งหมายความว่าเกือบสามในสี่ของ Bug ID ไม่สามารถดำเนินการกระบวนการทดสอบจนได้ผลลัพธ์การเปรียบเทียบที่สมบูรณ์ระหว่างรุ่น buggy และ fixed ซึ่งมีปัจจัยเกี่ยวเนื่องมาจากขั้นตอนคอมไพล์ การประมวลผลรันเทสต์ หรือการวัดค่า Coverage อย่างไรก็ดี ไม่ได้แปลว่าเคสทดสอบย่อยทั้งหมดของ Bug ID เหล่านั้นจะใช้งานไม่ได้

ปัจจัยดังกล่าวทำให้เกิดช่องว่างความแตกต่างอย่างเห็นได้ชัดระหว่าง Measured Coverage กับ Overall Coverage โดยส่งผลให้ค่าเฉลี่ย Mean Line Coverage ลดลงจาก 73.94% เหลือเพียง 18.70% และค่าเฉลี่ย Mean Condition Coverage ลดลงจาก 66.09% เหลือเพียง 16.48% เมื่อพิจารณาในภาพรวมประเมินผล

ดังนั้น ถึงแม้ว่า DeepSeek จะมีศักยภาพที่สูงมากด้านการทำ Coverage ในกรณีที่วัดผลสำเร็จ แต่การแปลผลดังกล่าวต้องนำสัดส่วน NOT_AVAILABLE ที่อยู่ในระดับสูงมาประกอบการพิจารณาด้วย เนื่องจากตัวเลข Coverage ที่จัดเก็บได้เป็นเพียงข้อมูลจากส่วนหนึ่งของ Bug ID ทั้งหมดในการทดลอง

3. GA (Genetic Algorithm)

ข้อดี

Genetic Algorithm สามารถทำระดับความครอบคลุมของโค้ดได้โดดเด่น โดยบันทึกค่าเฉลี่ย Mean Line Coverage (Measured) ที่ 65.02% และ Mean Condition Coverage (Measured) ที่ 61.37% ซึ่งเป็นรองเพียง DeepSeek เท่านั้น และเมื่อพิจารณาในแง่ Overall Coverage กลับพบว่า GA ทำผลงานได้สูงสุด โดยมีค่าเฉลี่ย Mean Line Coverage ที่ 59.02% และ Mean Condition Coverage ที่ 51.70%

อัลกอริทึม GA มีสัดส่วน NOT_AVAILABLE อยู่ที่ 7.61% ซึ่งต่ำกว่าโมเดลภาษาอย่าง Gemini และ DeepSeek อย่างมีนัยสำคัญ บ่งบอกว่าขั้นตอนตั้งแต่การสร้างชุดทดสอบ คอมไพล์รัน ไปจนถึงการวัด Coverage มีความราบรื่นและสามารถประมวลผลจนได้ผลการเปรียบเทียบระหว่างรุ่น buggy และ fixed สำหรับ Bug ID ส่วนใหญ่

นอกจากนี้ยังพบอัตรา FAIL เพียง 0.12% ซึ่งแสดงว่ามีเพียงส่วนน้อยมากที่ตัวอัลกอริทึมไม่สามารถสร้างชุดทดสอบขึ้นมาประมวลผลต่อได้

ข้อจำกัด

แม้ว่า GA จะมีระดับ Coverage สูงพร้อมทั้งมีสัดส่วน NOT_AVAILABLE ที่ค่อนข้างต่ำ แต่กลับไม่พบกรณีที่ เป็น REVEALING เลย (0.00%) ในขณะที่มีอัตราส่วน NOT_REVEALING สูงถึง 70.26% แสดงว่า Bug ID ส่วนใหญ่ที่ประเมินผลผ่านนั้นกลับให้ผลการรันผ่านทั้งบนเวอร์ชัน buggy และ fixed จึงไม่สามารถระบุตำแหน่ง Defect เป้าหมายได้

ข้อสรุปนี้ชี้ให้เห็นว่า GA มีประสิทธิภาพด้านการค้นหา Input เพื่อครอบคลุมโครงสร้างโปรแกรมและเพิ่ม Coverage แต่ค่า Coverage ที่สูงก็ไม่ได้รับประกันความสามารถในการตรวจพบข้อผิดพลาด เนื่องจากเคสทดสอบอาจเข้าถึงบรรทัดโค้ดที่มีข้อบกพร่อง แต่เงื่อนไข assertion ไม่มีความคมชัดพอที่จะจำแนกความแตกต่างระหว่างพฤติกรรมในรุ่น buggy และ fixed ได้

4. Feedback-Directed Random Test Generation

ข้อดี

Feedback-Directed Random Test Generation มีสัดส่วน NOT_AVAILABLE ต่ำที่สุดในการทดลองโดยคิดเป็นเพียง 2.40% บ่งชี้ว่ากระบวนการประเมินสามารถดำเนินไปจนได้ผลลัพธ์การเปรียบเทียบระหว่างรุ่น buggy และ fixed ได้ครบถ้วนสำหรับ Bug ID ส่วนใหญ่

แนวทางนี้จึงมีความเสถียรในขั้นตอนการสร้างและประมวลผลชุดทดสอบอย่างสม่ำเสมอ และเหมาะสำหรับใช้เป็น Baseline ในการเปรียบเทียบกับแนวทางอื่นๆ ทั้งยังปรากฏสัดส่วน REVEALING ที่ 0.12% ซึ่งแสดงว่ายังคงค้นพบ Defect เป้าหมายได้ในบางกรณี

ข้อจำกัด

แนวทาง Feedback-Directed มีระดับ Coverage ต่ำที่สุดในส่วน Measured Coverage โดยทำค่าเฉลี่ย Mean Line Coverage ที่ 35.05% และ Mean Condition Coverage ที่ 24.92% ในขณะที่ค่าเฉลี่ย Overall Coverage อยู่ที่ 33.78% และ 24.09% ตามลำดับ

นอกจากนี้ยังมีอัตราส่วน NOT_REVEALING สูงถึง 65.81% บ่งบอกว่า Bug ID ส่วนใหญ่ประมวลผลผ่านได้ทั้งรุ่น buggy และ fixed ทำให้ไม่สามารถเปิดเผยข้อผิดพลาดเป้าหมายได้ อีกทั้งมีสัดส่วน INCONCLUSIVE สูงที่สุดถึง 29.51% ซึ่งว่ามีหลายกรณีที่ประมวลผลบนสองเวอร์ชันได้สำเร็จ แต่ผลลัพธ์จาก assertion ยังไม่ชัดเจนพอที่จะยืนยัน Defect ได้

ยิ่งไปกว่านั้น แนวทางนี้มีอัตรา FAIL อยู่ที่ 2.17% ซึ่งสูงที่สุดจากทั้ง 4 เครื่องมือ หมายความว่า มี Bug ID บางรายการที่ตัวเครื่องมือล้มเหลวในขั้นตอนการเจนเนอเรตโค้ดจนไม่สามารถสร้างไฟล์ทดสอบมาประมวลผลต่อได้

4.7 สรุปผลการเปรียบเทียบ

จากการเปรียบเทียบเครื่องมือทั้ง 4 แนวทาง ได้แก่ Gemini 3.8 Flash, DeepSeek v4 Flash, Genetic Algorithm และ Feedback-Directed Random Test Generation พบว่าแต่ละแนวทางต่างมีจุดเด่นและข้อจำกัดเฉพาะตัว โดยไม่มีแนวทางใดให้ผลลัพธ์ที่ดีที่สุดในทุกมิติตัวชี้วัด ซึ่งสามารถสรุปภาพรวมได้ดังนี้

1) ด้าน Code Coverage

เมื่อพิจารณา Measured Coverage ซึ่งประเมินเฉพาะกรณีที่ดำเนินการวัดผลสำเร็จ พบว่า DeepSeek v4 Flash มีค่าสูงสุด โดยมี Mean Line Coverage ที่ 73.94% และ Mean Condition Coverage ที่ 66.09% ตามมาด้วย Genetic Algorithm (65.02% และ 61.37%), Gemini (49.00% และ 43.76%) และ Feedback-Directed (35.05% และ 24.92%) ตามลำดับ

อย่างไรก็ดี เมื่อประเมิน Overall Coverage ซึ่งสะท้อนผลการทดลองของทุก Bug ID ในภาพรวม พบว่า Genetic Algorithm ให้ผลลัพธ์สูงสุด โดยมี Mean Line Coverage ที่ 59.02% และ Mean Condition Coverage ที่ 51.70% ขณะที่อันดับถัดมาในส่วน Line Coverage ได้แก่ Feedback-Directed (33.78%), Gemini (28.39%) และ DeepSeek (18.70%)

ในมิติ Condition Coverage แบบ Overall ตัว GA ยังคงครองอันดับสูงสุดที่ 51.70% ตามด้วย Gemini 25.27%, Feedback-Directed 24.09% และ DeepSeek 16.48%

ความแตกต่างระหว่าง Measured Coverage และ Overall Coverage ปรากฏให้เห็นชัดเจนใน DeepSeek เนื่องจากมีสัดส่วน NOT_AVAILABLE สูง ดังนั้น การพิจารณา Coverage จึงต้องประเมินควบคู่กับความเสถียรของเครื่องมือในการรัน Bug ID ให้ผ่านขั้นตอนประเมินจนสามารถวัดผลได้จริง

2) ด้านความสามารถในการตรวจจับข้อบกพร่อง

Gemini 3.8 Flash ทำอัตราส่วน REVEALING ได้สูงสุดที่ 0.70% ตามมาด้วย DeepSeek v4 Flash ที่ 0.47%, Feedback-Directed Random Test Generation ที่ 0.12% และ Genetic Algorithm ที่ 0.00%

ผลลัพธ์นี้แสดงว่าในระดับ Bug ID ของแต่ละรอบ เครื่องมือประเภทโมเดลภาษาสามารถสร้าง Test Suite ที่เข้าเงื่อนไข REVEALING ได้มากกว่าแนวทางอัตโนมัติแบบเดิม กล่าวคือ สามารถทำให้การทดสอบล้มเหลวในรุ่น buggy แต่ประมวลผลผ่านในรุ่น fixed

อย่างไรก็ตาม สัดส่วน REVEALING ของทุกแนวทางยังคงต่ำกว่า 1% จึงยังไม่เพียงพอที่จะตัดสินว่าเครื่องมือใดมีสมรรถนะในการตรวจพบข้อผิดพลาดที่เหนือกว่าอย่างชัดเจนหากพิจารณาเพียงตัวชี้วัดนี้ลำพัง

3) ด้านความสามารถในการดำเนินการกระบวนการประเมินจนสมบูรณ์

Feedback-Directed Random Test Generation มีสัดส่วน NOT_AVAILABLE ต่ำที่สุดที่ 2.40% ตามด้วย Genetic Algorithm ที่ 7.61% ในขณะที่ Gemini และ DeepSeek มีสัดส่วนสูงชันมากที่สุดที่ 42.51% และ 75.06% ตามลำดับ

สถานะ NOT_AVAILABLE ในงานวิจัยนี้ได้แปลว่าเคสทดสอบย่อยรันไม่ได้ทั้งหมด แต่หมายถึงในระดับ Bug ID ต่อรอบ ไม่สามารถรันจนเกิดผลการเปรียบเทียบที่สมบูรณ์ระหว่าง buggy และ fixed ซึ่งมีสาเหตุจากปัญหาคอมไพล์ การประมวลผล การจัดเก็บ Coverage หรือรันผ่านได้เพียงรุ่นเดียว

ดังนั้น ผลการทดลองจึงชี้ว่า GA และ Feedback-Directed สามารถพา Bug ID เข้าสู่การประเมินผลได้ครบถ้วนในสัดส่วนที่สูงกว่าเครื่องมือ AI โดยเฉพาะ DeepSeek ที่แม้จะมี Measured Coverage สูง แต่ก็มี Bug ID จำนวนมากที่ไม่สามารถผ่านเข้าสู่การประเมินขั้นสุดท้ายได้

4) ด้านผลลัพธ์จาก Assertion

Genetic Algorithm และ Feedback-Directed มีอัตราส่วน NOT_REVEALING อยู่ในระดับสูงที่ 70.26% และ 65.81% ตามลำดับ สะท้อนว่า Bug ID จำนวนมาก แม้จะประมวลผลผ่านได้ทั้งสองเวอร์ชัน แต่การที่ชุดทดสอบรันผ่านทั้งหมดทำให้ไม่สามารถตรวจจับ Defect เป้าหมายได้

ในส่วนของ Gemini มี NOT_REVEALING อยู่ที่ 29.39% และ DeepSeek อยู่ที่ 4.33% ทว่าไม่อาจสรุปว่า DeepSeek มีคุณภาพ assertion ที่ดีกว่าได้ทันที เนื่องจาก DeepSeek มีสัดส่วน NOT_AVAILABLE สูงถึง 75.06% ส่งผลให้ Bug ID จำนวนมากยังไม่เข้าสู่ขั้นตอนที่จะจำแนกผลลัพธ์จาก assertion ได้ครบถ้วน

สำหรับกรณี INCONCLUSIVE พบว่า Feedback-Directed มีสัดส่วนสูงสุดที่ 29.51% ตามมาด้วย Gemini 27.17%, GA 22.01% และ DeepSeek 20.14% โดยหมายถึงกรณีที่รันผ่านได้ทั้งสองเวอร์ชันแต่ผล assertion ยังไม่สามารถยืนยัน Defect ได้อย่างชัดเจน เช่น ล้มเหลวทั้งสองรุ่น หรือรันผ่านบน buggy แต่ล้มเหลวบน fixed

5) ด้านความสามารถในการสร้าง TestCode

Gemini และ DeepSeek ปรากฏค่า FAIL ที่ 0% แสดงว่าไม่มี Bug ID ใดที่ไม่เคสล้มเหลวในการสร้าง TestCode สำหรับส่งเข้าสู่กระบวนการถัดไป ในขณะที่ Genetic Algorithm มี FAIL อยู่ที่ 0.12% และ Feedback-Directed อยู่ที่ 2.17%

อย่างไรก็ตาม จำเป็นต้องแยกแยะสถานะ FAIL ออกจาก NOT_AVAILABLE ให้ชัดเจน เพราะ FAIL หมายถึงความล้มเหลวตั้งแต่ขั้นตอนเจนเนอเรต TestCode ในขณะที่ NOT_AVAILABLE หมายถึงสร้างโค้ดได้แล้ว แต่นำไปประมวลผลวัดผลเปรียบเทียบระหว่าง buggy และ fixed ได้ไม่สมบูรณ์

กล่าวโดยสรุป DeepSeek แสดงศักยภาพด้าน Coverage ได้สูงที่สุดในกลุ่ม Bug ID ที่วัดผลสำเร็จ ในขณะที่ Gemini มีสัดส่วน REVEALING สูงสุด ส่วน Genetic Algorithm ทำได้ดีในด้าน Overall Coverage และนำ Bug ID เข้าสู่การประเมินได้ในสัดส่วนสูง และ Feedback-Directed Random Test Generation มี NOT_AVAILABLE ต่ำที่สุด แม้จะมีค่า Coverage และ REVEALING ค่อนข้างต่ำ

ผลการทดลองจึงสะท้อนว่า การประเมินเครื่องมือสร้างชุดทดสอบไม่อาจพิจารณาเพียง Coverage หรืออัตรา REVEALING มิติใดมิติหนึ่ง แต่ต้องประเมินควบคู่กันทั้งความสามารถในการสร้าง TestCode ความเสถียรในการประมวลผลจนสมบูรณ์ ระดับ Code Coverage และประสิทธิภาพของ assertion เพื่อให้ครอบคลุมจุดเด่นและข้อจำกัดของแต่ละแนวทางได้อย่างแท้จริง

บทที่ 5

สรุปผล ปัญหา และข้อเสนอแนะ

5.1 สรุปผลการทดลอง

จากการเปรียบเทียบประสิทธิภาพในการสร้างชุดทดสอบระหว่าง AI-Assisting Tools (Gemini 3.8 Flash และ DeepSeek v4 Flash) กับ Automated Test Generation Tools (Genetic Algorithm และ Feedback-Directed Random Test Generation) พบว่าแต่ละแนวทางต่างมีจุดเด่นและข้อจำกัดที่ไม่เหมือนกัน โดยไม่มีเครื่องมือใดแสดงผลลัพธ์ที่ดีที่สุดเหนือกว่าในทุกมิติตัวชี้วัด

ในส่วนของ Code Coverage หากพิจารณาเฉพาะกรณีประมวลผลสำเร็จหรือ Measured Coverage จะพบว่า DeepSeek v4 Flash มีประสิทธิภาพสูงสุดด้วยค่าเฉลี่ย Mean Line Coverage 73.94% และ Mean Condition Coverage 66.09% สะท้อนว่าโมเดลดังกล่าวสามารถสร้างชุดทดสอบที่เข้าถึงโครงสร้างและเงื่อนไขของโปรแกรมได้อย่างครอบคลุม อย่างไรก็ตาม DeepSeek กลับปรากฏสัดส่วน NOT_AVAILABLE สูงถึง 75.06% ส่งผลให้ค่า Overall Coverage ในภาพรวมลดลงเหลือเพียง Line Coverage 18.70% และ Condition Coverage 16.48%

ในทางตรงกันข้าม Genetic Algorithm ซึ่งมีค่า Measured Coverage เป็นอันดับสอง รองจาก DeepSeek กลับทำ Overall Coverage ได้สูงสุดในภาพรวม โดยมี Mean Line Coverage 59.02% และ Mean Condition Coverage 51.70% เนื่องจากสามารถนำ Bug ID ส่วนใหญ่เข้าสู่การประเมินจนได้ผลลัพธ์ โดยพบ NOT_AVAILABLE เพียง 7.61% ส่วน Feedback-Directed Random Test Generation แม้จะมีสัดส่วน NOT_AVAILABLE ต่ำที่สุดที่ 2.40% แต่กลับให้ค่าความครอบคลุมต่ำกว่าแนวทางอื่นอย่างมีนัยสำคัญ

สำหรับความสามารถในการเปิดเผย Defect หรือ REVEALING พบว่า Gemini 3.8 Flash มีสัดส่วนสูงสุดที่ 0.70% ตามด้วย DeepSeek v4 Flash (0.47%) และ Feedback-Directed Random Test Generation (0.12%) ขณะที่ Genetic Algorithm ไม่พบกรณี REVEALING ในการทดลองนี้ ทั้งนี้ อัตรา REVEALING ของทุกเครื่องมือยังคงต่ำกว่า 1% จึงไม่เพียงพอที่จะนำมาใช้สรุปประสิทธิภาพการตรวจจับข้อผิดพลาดได้อย่างเด็ดขาด

ด้านผลลัพธ์จาก assertion ปรากฏว่า Genetic Algorithm และ Feedback-Directed Random Test Generation มีสัดส่วน NOT_REVEALING สูงถึง 70.26% และ 65.81% ตามลำดับ บ่งชี้ว่าการทดสอบส่วนใหญ่รันผ่านทั้งบนเวอร์ชัน buggy และ fixed โดยไม่สามารถชี้จุดแตกต่างได้ ส่วน Gemini และ DeepSeek แม้จะมีอัตรา REVEALING ที่สูงกว่า แต่ต้องเผชิญกับอัตราส่วน NOT_AVAILABLE ที่ค่อนข้างมาก โดยเฉพาะ DeepSeek

ผลการวิเคราะห์ชี้ให้เห็นว่า ตัวเลข Coverage ที่สูงไม่ได้การันตีความสามารถในการตรวจจับ Bug ได้เสมอไป เนื่องจากชุดทดสอบอาจครอบคลุมบรรทัดโค้ดแต่ขาด assertion ที่รัดกุมพอในการตรวจวัดพฤติกรรม ขณะเดียวกัน เครื่องมือที่มีแนวโน้มเป็น REVEALING สูงกว่า กลับยังติดขัดอุปสรรคในกระบวนการคอมไพล์ รันเทสต์ หรือการวัด Coverage ทำให้ไม่สามารถประเมินผล Bug ID นั้นได้อย่างสมบูรณ์

สรุปได้ว่า Automated Test Generation Tools มีจุดแข็งด้านความเสถียรของกระบวนการประเมินและเหมาะสำหรับการสำรวจเส้นทางเพื่อเพิ่ม Code Coverage โดยเฉพาะ GA ที่ให้ผลลัพธ์ Overall Coverage สูงสุด กระนั้น ชุดทดสอบส่วนใหญ่ยังคงตกอยู่ในกลุ่ม NOT_REVEALING ซึ่งต้องได้รับการปรับปรุงคุณภาพ assertion เพื่อเพิ่มสมรรถนะการตรวจจับความผิดพลาดต่อไป

ขณะเดียวกัน AI-Assisting Tools แสดงศักยภาพในการระบุพฤติกรรมความแตกต่างระหว่างเวอร์ชันได้อย่างน่าสนใจ โดย Gemini และ DeepSeek มีอัตรา REVEALING เหนือกว่ากลุ่ม Automated Tools และ DeepSeek สามารถทำ Measured Coverage ได้สูงสุด อย่างไรก็ตาม ยังคงมีข้อจำกัดด้านความสมบูรณ์ของกระบวนการทดสอบ ซึ่งสะท้อนผ่านค่า NOT_AVAILABLE ที่พุ่งสูงอันเนื่องมาจากปัญหาในขั้นตอนการคอมไพล์และการประมวลผล

จากข้อสังเกตข้างต้น แนวทางการประยุกต์ใช้แบบผสมผสาน (Hybrid Approach) จึงถือเป็นทางออกที่มีประสิทธิภาพ โดยอาศัย Automated Tools ในการสร้าง input และเจาะสำรวจพื้นที่โค้ดให้ครอบคลุม ควบคู่กับการใช้ AI หรือผู้ทดสอบในการปรับปรุง assertion และคาดการณ์พฤติกรรมระบบ แทนที่จะเลือกใช้เพียงวิธีใดวิธีหนึ่ง

5.2 ปัญหาและอุปสรรคในการทดลอง

5.2.1 ปัญหาด้านสภาพแวดล้อม

กระบวนการทดลองดำเนินงานบนสภาพแวดล้อม Linux ผ่าน WSL โดยอาศัย Defects4J ระบบ Java และสคริปต์อัตโนมัติในการเตรียมโปรเจกต์ สร้างชุดทดสอบ และประมวลผล จึงจำเป็นต้องมีการตั้งค่าอุปกรณ์และสภาพแวดล้อมต่างๆ ให้พร้อมก่อนเริ่มทดสอบจริง

เนื่องจากต้องรองรับหลายโปรเจกต์และ Bug ID จำนวนมาก สคริปต์จึงต้องทำ checkout ซอร์สโค้ดตามเวอร์ชันที่กำหนด พร้อมคอมไพล์และตั้งค่า dependency สำหรับแต่ละเป้าหมายซ้ำกันหลายรอบ ส่งผลให้การบริหารจัดการสภาพแวดล้อมและการตรวจสอบสถานะความพร้อมกลายเป็นขั้นตอนสำคัญอย่างยิ่ง

นอกจากนี้ ความแตกต่างของโครงสร้างโปรเจกต์ ไลบรารีอ้างอิง และระดับเวอร์ชัน Java ในแต่ละเป้าหมาย ได้สร้างอุปสรรคต่อการคอมไพล์และการรัน เทสต์ จึงต้องมีการบันทึกสถานะการทำงานอย่างเป็นระบบเพื่อวิเคราะห์สาเหตุของข้อผิดพลาดที่ไม่สามารถประเมินผลได้อย่างสมบูรณ์

5.2.2 ปัญหาในการสร้าง Test Case ด้วย AI

การใช้ AI-Assisting Tools จำเป็นต้องส่งซอร์สโค้ดคลาสเป้าหมายและบริบทที่เกี่ยวข้องไปยังโมเดลวิเคราะห์ ส่งผลให้ปริมาณ token และระยะเวลาที่ใช้ในการเจนเออเรตแปรผันตามขนาดและความซับซ้อนของแต่ละไฟล์

ตัวอย่างจากการทดลองกับ DeepSeek บนไฟล์ NodeUtil.java ต้องใช้ prompt สูงถึง 17,059 token และกินเวลาสร้างนานราว 301 วินาที ซึ่งเห็นว่า คลาสขนาดใหญ่ต้องใช้เวลาและทรัพยากรสูงชันอย่างมาก และเมื่อขยายผลไปยัง Bug ID จำนวนมาก ระยะเวลารวมในการสร้างจึงสะสมเพิ่มขึ้นตามลำดับ

ยิ่งไปกว่านั้น การเรียกใช้งานผ่าน API ยังมีความเสี่ยงจากปัจจัยภายนอก เช่น จากการรัน Gemini พบปัญหากระบวนการชะงัก 4 รายการ ซึ่งมีสาเหตุมาจาก Gateway TTFT Timeout 3 รายการ และขีดข้องด้านโควตา API Key หรือการเชื่อมต่ออีก 1 รายการ

จากปัญหาดังกล่าว จึงต้องมีการพัฒนาระบบบันทึกสถานะอย่างละเอียด เพื่อแยกแยะความล้มเหลวที่เกิดจากข้อผิดพลาดของการเชื่อมต่อ API ออกจากความสามารถในการสร้าง Test Case ของตัวโมเดลอย่างชัดเจน

ประเด็นสำคัญอีกประการคือ แม้โมเดลจะเจนเออเรต TestCode ได้สำเร็จ แต่โค้ดดังกล่าวอาจพบข้อผิดพลาดในขั้นตอนถัดไป เช่น คอมไพล์ไม่ผ่าน หรือเรียกใช้ API ที่ไม่มีอยู่จริง จนส่งผลให้สัดส่วน NOT_AVAILABLE ของเครื่องมือ AI พุ่งสูงขึ้น

5.2.3 ปัญหาในการสร้าง Test Case ด้วย Automated Tools

เครื่องมืออัตโนมัติอย่าง EvoSuite และ Randoop ต้องอาศัยการเตรียมโปรเจกต์เป้าหมายให้สามารถคอมไพล์ได้ก่อนเริ่มค้นหาชุดทดสอบ โดยมีขั้นตอนตั้งแต่การ checkout รุ่น buggy, การคอมไพล์, การรันเครื่องมือตามเวลาที่กำหนด ไปจนถึงการบันทึกไฟล์ Java ที่ได้

เมื่อต้องดำเนินการซ้ำกับ Bug ID จำนวนมาก กระบวนการดังกล่าวกินทรัพยากรทั้ง CPU, หน่วยความจำ และพื้นที่จัดเก็บไฟล์ build เป็นอย่างมาก

แม้ Automated Tools จะมีอัตรา NOT_AVAILABLE ต่ำกว่า AI แต่ผลลัพธ์ส่วนใหญ่ของ GA และ Feedback-Directed กลับกระจุกตัวอยู่ในกลุ่ม NOT_REVEALING แสดงว่าการรันเทสต์ผ่านไม่ได้หมายความว่าสามารถตรวจจับ Defect ได้โดยอัตโนมัติ

ข้อจำกัดหลักจึงขึ้นอยู่กับความสามารถในการสร้าง assertion เพื่อแยกแยะความแตกต่างระหว่างรุ่น buggy และ fixed มากกว่าแค่การสร้าง input หรือเพิ่ม Code Coverage เพียงอย่างเดียว

5.2.4 ข้อจำกัดของเครื่องมือและการทดลอง

เครื่องมือแต่ละประเภทมีข้อจำกัดที่ต่างกัน โดย AI-Assisting Tools ได้รับผลกระทบจากข้อจำกัดของโมเดลภาษา เช่น ปริมาณ token, เวลาตอบสนอง, Gateway Timeout และการอ้างอิง API ที่ไม่ตรงกับสภาพแวดล้อมจริง ส่งผลให้เกิด NOT_AVAILABLE สูง

ในส่วนของ EvoSuite และ Randoop ต้องพึ่งพาสภาพแวดล้อมที่คอมไพล์สมบูรณ์ และใช้ทรัพยากรเครื่องสูงเมื่อต้องประมวลผลหลายเป้าหมายพร้อมกัน

นอกจากนี้ สัดส่วน REVEALING ที่ต่ำกว่า 1% ในทุกเครื่องมือ ทำให้การเปรียบเทียบความสามารถในการตรวจจับ Bug ต้องทำอย่างระมัดระวัง และไม่สามารถใช้ตัดสินประสิทธิภาพได้โดยลำพัง

ประการสุดท้าย การประเมินในระดับ 1 Bug ต่อ 1 รอบ ทำให้สถานะ NOT_AVAILABLE หมายถึงความล้มเหลวของกระบวนการประเมินใน Bug ID นั้นทั้งหมด แต่ไม่ได้หมายความว่ากรณีทดสอบย่อยทั้งหมดจะใช้งานไม่ได้

5.3 สิ่งที่ได้เรียนรู้และแนวทางพัฒนาในอนาคต

การวัดประสิทธิภาพเครื่องมือสร้างชุดทดสอบจำเป็นต้องพิจารณาหลายมิติร่วมกัน ทั้ง Code Coverage, ปริมาณ Test Case และการประมวลผลจริง เนื่องจากเป้าหมายหลักไม่ได้มีเพียงการเข้าถึงโค้ด แต่รวมถึงการตรวจสอบความถูกต้องของพฤติกรรมระบบด้วย

กรณีของ GA ชี้ให้เห็นว่า Coverage ที่สูงที่สุดก็อาจไม่พบ REVEALING เลย ในขณะที่ DeepSeek ทำ Measured Coverage ได้สูงสุด แต่กลับมี NOT_AVAILABLE สูง ดังนั้นการประเมินจึงต้องพิจารณา Coverage ควบคู่กับความเสถียรของกระบวนการรันเสมอ

นอกจากนี้ คุณภาพของ assertion ถือเป็นหัวใจสำคัญ เพราะหาก assertion ไม่ครอบคลุม พฤติกรรมที่ผิดพลาดก็จะไม่ถูกตรวจพบ ส่งผลให้ได้สถานะ NOT_REVEALING การพัฒนาในอนาคตจึงต้องเน้นการปรับปรุง assertion ควบคู่ไปกับการสร้าง input

สำหรับแนวทางการพัฒนาในอนาคต มีประเด็นที่ควรมุ่งเน้นดังนี้

1. การปรับปรุงกระบวนการสร้าง Test Case ด้วย AI โดยการพัฒนา prompt และ context ที่ระบุข้อมูลโครงสร้างคลาสและ API ให้ครบถ้วน เพื่อลดข้อผิดพลาดของโค้ดทดสอบ รวมถึงสร้างระบบแก้ไขโค้ดอัตโนมัติ
2. การวิเคราะห์สาเหตุของ NOT_AVAILABLE ให้ละเอียดยิ่งขึ้น โดยแยกแยะประเภทข้อผิดพลาด เช่น คอมไพล์ไม่ผ่าน หรือรันได้เพียงเวอร์ชันเดียว เพื่อระบุจุดบกพร่องที่แท้จริงของแต่ละเครื่องมือ
3. การพัฒนา assertion ของ Automated Tools โดยการเพิ่มคุณภาพการตรวจสอบพฤติกรรมผ่านการนำ specification หรือข้อมูลจากรุ่น fixed มาช่วยวิเคราะห์
4. การศึกษาแนวทาง Hybrid Test Generation โดยรวมจุดเด่นของการสร้าง input จาก Automated Tools เข้ากับการสร้าง assertion ที่ฉลาดจาก AI
5. การเพิ่มจำนวนรอบและการวิเคราะห์ทางสถิติ เพื่อให้ผลลัพธ์การตรวจจับข้อผิดพลาดที่มีสัดส่วนต่ำมีความน่าเชื่อถือยิ่งขึ้น
6. การพัฒนาระบบจัดการข้อผิดพลาดและการทำงานซ้ำอัตโนมัติ เช่น กลไก retry สำหรับ API และการบริหารจัดการไฟล์ชั่วคราวของ Automated Tools

สรุปได้ว่า คุณภาพของชุดทดสอบไม่ได้ขึ้นอยู่กับปริมาณหรือ Coverage เพียงอย่างเดียว แต่ต้องอาศัยความสัมพันธ์ระหว่างการครอบคลุมโค้ด ความเสถียรในการประมวลผล และความแม่นยำของ assertion ในการตรวจจับข้อผิดพลาด

เอกสารอ้างอิง

A. Krafczyk and K. Schmid, “Reproducible Automated Program Repair Is Hard – Experiences With the Defects4J Dataset,” *arXiv preprint arXiv:2604.26674*, Apr. 2026.

C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, “Feedback-directed random test generation,” in *Proc. 29th International Conference on Software Engineering (ICSE 2007)*, Minneapolis, MN, USA, 2007, pp. 75–84, doi: 10.1109/ICSE.2007.37.

J. M. Rojas, G. Fraser, and A. Arcuri, “Automated Unit Test Generation during Software Development: A Controlled Experiment and Think-Aloud Observations,” in *Proc. 2015 International Symposium on Software Testing and Analysis (ISSTA '15)*, 2015, pp. 338–349, doi: 10.1145/2786805.2786828.